

CSC 648 (Section 3)

Vibe Check: Milestone 2

ARKHA (Team 6)

March 4th, 2026

Harry Kakadiya (Team Lead): hkakadiya@sfsu.edu

Kaitlyn Ashby (Github master, Technical writer): kashby@sfsu.edu

Rahul Srinath (Software Architect): rsrinath@sfsu.edu

Amulya Sagi (Backend Lead): asagi@sfsu.edu

Aljhay Soriano (Scrum Master): asoriano6@sfsu.edu

<https://github.com/sfsu-joseo/csc648-848-project-sp26-ARKHA>

Revision	Description
v3	2/27 @ 5:22pm: Milestone 2, Version 1 Draft
v2	2/24 @ 9:03 pm: Finished Milestone 1, checkpoint 3 requirements
v1	Created document 2/7 @ 5:22pm, added tech stack for Care Companion and Vibe Check apps

Table of Contents

Data Definitions	3
User	3
Location	4
Note	4
Category Tag	4
Reply	4
Reaction	4
Heatmap Activity	4
User Profile	4
Notification	4
Friend	5
Vibe Score	5
Badge	5
Functional Requirements	5
Priority 1	5
Authentication	5
Location	5
Heatmap	5
Notes	6
Vibe Score	6
Priority 2	6
Heatmap	6
Notes	6
Search/Filter	7
Vibe Score	7
Profile Management	7
Moderation	7
Priority 3	7
Notification	7
Friends	7
Badge	8
Profile Management	8
Mockups and Storyboards	9
System Design	9

Database Architecture	9
Backend Architecture	9
Network and Deployment Design	9
High Level APIs and Algorithms	9
Key Project Risks	9
Project Management	9
Team Contributions	9

Data Definitions

User

Definition: A user is a person who uses the application to view nearby locations and interact with content. A User can create Notes and can reply to or react to existing Notes.

Attribute	Description
user_id	Unique identifier for each User.
username	Public display name shown on Notes and the User Profile.
email	Email address used for login and system notifications.
password	Plaintext version of password that registered user sign in with, authenticated against password_hash
password_hash	Securely hashed password. Never stored in plaintext.
profile_picture_url	URL pointing to the user's profile photo.
created_at	Timestamp of when the account was created.
account_status	Current state of the account: active, suspended, or deleted.

Registered User

Definition: A Registered User is a User who has completed the full account registration process by verifying their email and setting a password. Only Registered Users are permitted to create Notes, post Replies, submit Reactions, and access social features such as Friends and Badges. A guest or unverified User becomes a Registered User upon successful email verification.

Attribute	Description
reg_user_id	Unique identifier for the Registered User record.
user_id	Foreign key linking to the base User entity. Each Registered User corresponds to exactly one User.
email_verified	Boolean flag indicating whether the user has confirmed their email address. Must be true for full platform access.
registration_completed_at	Timestamp of when the user completed the full registration process.
role	The permission level of the registered user: standard or admin. Default is standard.

Location

Definition: A Location represents a physical place shown on the map (e.g., café, library, venue, park). Notes are tied to a specific Location to ensure that updates are relevant to that place.

Attribute	Description
location_id	Unique identifier for each Location.
name	Display name of the location (e.g., J. Paul Leonard Library, Philz Coffee).
address	Human-readable street address used for display and search.
latitude	GPS latitude coordinate. Used for map rendering and radius enforcement.
longitude	GPS longitude coordinate. Used for map rendering and radius enforcement.
category_tags	One or more category labels assigned to this location (e.g., Restaurant, Study Spot, Venue, Park).
avg_vibe_score	Rolling average of Vibe Scores from non-expired Notes at this location.

Note

Definition: A Note is a short, location-locked post created by a User to describe what is happening at a Location in real time. Each Note has a timestamp and an expiration time so that outdated information is automatically removed.

Attribute	Description
note_id	Unique identifier for each Note.
user_id	The User who posted the Note. Links to the User entity.
location_id	The Location this Note describes. Links to the Location entity.
content	The text of the Note. Maximum 280 characters.
vibe_score	The atmosphere rating selected by the user: Dead, Quiet, Moderate, Busy, or Buzzing.
is_anonymous	Whether the poster's username is hidden. Default is false.
created_at	Timestamp of when the Note was posted.
expires_at	Timestamp of when the Note auto-deletes. Typically 2–4 hours after created_at.

Category Tag

A Category Tag is a label used to classify a Note or Location (e.g., restaurant, study spot, venue line, parking). Category Tags allow users to filter and view relevant content more efficiently.

Attribute	Description
tag_id	Unique identifier for each Category Tag.
tag_name	Display label for the tag (e.g., Restaurant, Study Spot, Venue, Parking, Park).

Reply

A Reply is a response to a specific Note that provides additional context or updated information. Each Reply belongs to one Note.

Attribute	Description
reply_id	Unique identifier for each Reply.
note_id	The Note this Reply belongs to.
user_id	The User who posted the Reply.
content	Text content of the Reply. Maximum 280 characters.
created_at	Timestamp of when the Reply was posted.

Reaction

A Reaction represents a quick feedback action (such as an upvote or similar interaction) that a User can apply to a Note. Reactions help indicate which notes are useful or accurate.

Attribute	Description
reaction_id	Unique identifier for each Reaction.
note_id	The Note being upvoted.
user_id	The User who submitted the Reaction.
created_at	Timestamp of when the Reaction was submitted.

Heatmap Activity

Heatmap Activity is a visual representation of recent activity in a geographic area. It is generated from recent Notes and user interactions within a specific time window and helps users quickly assess how active an area is.

Attribute	Description
latitude	Center GPS latitude of the heatmap area cell.

longitude	Center GPS longitude of the heatmap area cell.
activity_score	Count of non-expired Notes in this area within the active time window. Higher score = more intense heat color.
time_window	The rolling time range used to compute activity. Default is 2 hours.
computed_at	When this heatmap snapshot was last calculated.

User Profile

A user profile stores user preferences such as notification settings, default radius, preferred categories. It tracks user contribution stats (for future gamification).

Attribute	Description
profile_id	Unique identifier for each User Profile.
user_id	The User this Profile belongs to. Each User has exactly one Profile.
default_radius_km	The default search and filter radius in kilometers. Default is 1.0 km.
preferred_categories	List of category tags the user prefers to see by default on the map.

visibility	Controls who can see this profile: public or friends only.
total_notes_posted	Lifetime count of Notes posted by this user. Used for badge tracking.
total_reactions_received	Lifetime count of upvotes received on the user's Notes. Used for badge tracking.
friend_notifications_enabled	Whether the user receives notifications when a friend leaves a nearby Note. Default is true.
notifications_enabled	Whether the user receives general system notifications. Default is true.

Notification

A notification represents a system notification (trending location nearby, etc.)

Attribute	Description
notification_id	Unique identifier for each Notification.
user_id	The User receiving this Notification.
type	The category of event: trending_location, friend_note, or badge_earned.

message	Human-readable notification text shown to the user.
is_read	Whether the user has viewed this notification. Default is false.
created_at	Timestamp of when the Notification was generated.

Friend

A Friend Connection represents a mutual relationship between two Users. It allows Users to see each other's Notes on the map and receive notifications when a friend leaves a Note nearby.

Attribute	Description
friendship_id	Unique identifier for each Friend Connection.
user_id_1	One side of the friendship.
user_id_2	Other side of the friendship.
status	Current state of the connection: pending, accepted, or blocked.
created_at	Timestamp of when the friend request was sent.

Vibe Score

A Vibe Score is a quick, standardized rating a User can apply to a Location through their Note, similar to Google Maps' thumbs system but framed around atmosphere. Instead of stars, users rate using descriptors (e.g., Quiet, Moderate, Buzzing) or a simple scale.

Attribute	Description
vibe_level	The descriptive label selected by the user: Dead, Quiet, Moderate, Busy, or Buzzing.
numeric_value	Internal numeric mapping used for aggregate calculations: Dead=1, Quiet=2, Moderate=3, Busy=4, Buzzing=5.
note_id	The Note this Vibe Score belongs to. Each Note has exactly one Vibe Score.

Badge

A Badge is an achievement awarded to a User for meeting specific contribution milestones (e.g., leaving 10 notes, visiting 5 cafes, being a top contributor in an area).

Attribute	Description
badge_id	Unique identifier for each Badge.

badge_name	Display name of the badge (e.g., Coffee Connoisseur, Local Legend).
description	What the user needs to do to earn this badge.
requirement_threshold	The count required to earn this badge (e.g., 10 notes at coffee shops).

Functional Requirements

Priority 1

Authentication

- FR 1.1: The user shall be able to create an account with an email and password.
- FR 1.2: The user shall be able to log in and log out.
- FR 1.3: The user shall be required to be authenticated in order to create notes, replies, or reactions.
- FR 1.4: The system shall send a verification email to the user upon account creation.
- FR 1.5: The system shall grant full platform access only after the user's email address has been verified.
- FR 1.6: The system shall display the user's registration status on their profile.
- FR 1.7: The system shall allow a user to resend the verification email if the original was not received.

Location

- FR 2.1: The user shall be able to grant permission for the app to access their current location.
- FR 2.2: The user shall be able to view nearby locations on the map.
- FR 2.3: The user shall be able to select a specific location from the map.

Heatmap

- FR 3.1: The user shall be able to view a heatmap showing activity in the nearby area.
- FR 3.3: The user shall be able to zoom in and out of the map.

Notes

- FR 4.1: The user shall be able to view notes related to a specific location.
- FR 4.7: The system shall enforce a limit of one reaction per user per note
- FR 8.9: The system shall hide notes and replies posted by blocked users from the blocking user's view.
- FR 4.9: The user shall be able to post a note at a specific location.
- FR 4.10: The user shall be able to compose text content when creating a note.
- FR 4.13: The system shall restrict note posting to users who are within the defined geographic radius of a location.
- FR 4.14: The system shall display a descriptive error message to the user when a note fails to post.
- FR 4.15: The user shall be able to view notes in a card format resembling a letterboxed review, showing vibe level, note content, timestamp, reaction count, and reply count.
- FR 4.17: The user shall be able to view the username or anonymous handle associated with each note.
- FR 4.18: The user shall be able to view the time elapsed since a note was posted.

Vibe Score

- FR 7.1: The user shall be able to select a vibe level (Dead, Quiet, Moderate, Busy, Buzzing) when creating a note.
- FR 7.2: The user shall be able to view the aggregate vibe score of a location based only on non-expired notes.
- FR 7.3: The system shall display a fallback message indicating limited data availability when fewer than the minimum required notes exist to calculate a vibe score.
- FR 7.4: The user shall be able to view vibe scores visually using descriptive labels rather than numerical star ratings.
- FR 7.5: The user shall be able to view an updated vibe score for a location when new notes are added or expired notes are removed.

Priority 2

Heatmap

- FR 3.2: The user shall be able to filter the heatmap by category.

Notes

- FR 4.2: The user shall be able to view replies to a note.
- FR 4.3: The user shall be able to reply to notes posted by other users.
- FR 4.4: The user shall be able to delete their own reply.
- FR 4.5: The user shall be able to view reactions to a note.
- FR 4.6: The user shall be able to react to notes posted by other users.
- FR 4.7: The user shall be able to react only once to the same note.
- FR 4.8: The user shall be able to remove a reaction.
- FR 4.11: The user shall be able to edit their own note.
- FR 4.12: The user shall be able to delete their own note.
- FR 4.16: The user shall be able to visually distinguish between notes posted by friends and notes posted by other users on the map.
- FR 4.19: The user shall be able to view the remaining time before a note expires.

Search/Filter

- FR 6.1: The user shall be able to search for locations by name or category.
- FR 6.2: The user shall be able to filter search results to a radius.
- FR 6.3: The user shall be able to filter search results by vibe level.

Vibe Score

- FR 7.6: The user shall be able to filter locations on the map by vibe level.

Moderation

- FR 11.1: The user shall be able to report a note for inappropriate content.
- FR 11.2: The user shall be able to provide a reason for reporting a note.
- FR 11.3: The user shall be able to report a reply for inappropriate content.
- FR 11.4: The user shall be able to provide a reason for reporting a reply.

User

- FR 10.1: The user shall be able to edit their username.
- FR 10.2: The user shall be notified if the username they want to change to is unavailable.
- FR 10.3: The user shall be able to change their password.
- FR 10.4: The user
- FR 10.5: The user shall be able to view their posting history.
- FR 10.6: The user shall be able to view another user's public profile
- FR 10.7: The user shall be able to view the username associated with notes, replies and reactions posted by other users.
- FR 10.8: The user shall be able to view their public user profile.

Notification

- FR 5.1: The user shall be able to receive a notification when a trendy location is nearby.
- FR 5.2: The user shall be able to enable or disable notifications in settings.

Priority 3

Friends

- FR 8.1: The user shall be able to send a friend request to another registered user.
- FR 8.2: The user shall be able to accept or decline an incoming friend request.
- FR 8.3: The user shall be able to remove a friend from their friends list.
- FR 8.4: The user shall be able to view a list of their current friends.
- FR 8.5: The user shall be able to see notes left by their friends on the map, visually distinguished from notes by non-friends.
- FR 8.6: The user shall be notified when a friend leaves a note at a nearby location.
- FR 8.7: The user shall be able to enable or disable friend activity notifications in settings.
- FR 8.8: The user shall be able to block another user, preventing them from viewing their notes or sending friend requests.
- FR 8.9: The user shall be able to view notes posted by only unblocked users.
- FR 8.10: The user shall be able to unblock a previously blocked user.
- FR 8.11: The user shall be able to see an indicator icon on map pins where a friend has recently left a note.

Badge

- FR 9.1: The user shall be automatically awarded badges when they meet defined contribution thresholds.

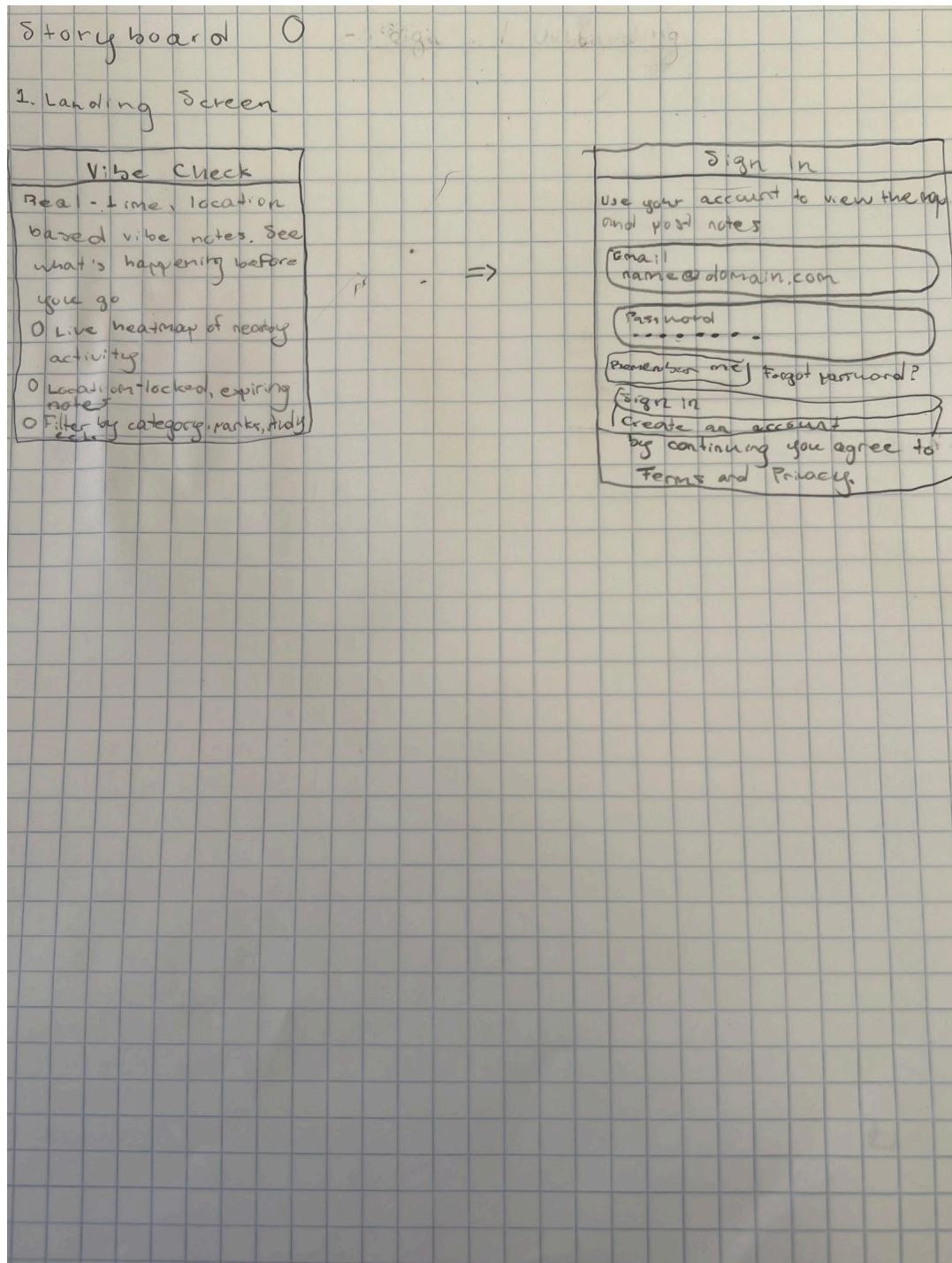
- FR 9.2: The user shall be able to view all badges they have earned on their profile.
- FR 9.3: The user shall be able to view all available badges and the requirements to earn them.
- FR 9.4: The user shall be notified when they earn a new badge.
- FR 9.5: The user shall be able to track the number of notes they have posted toward badge thresholds.
- FR 9.6: The user shall be able to track the number of unique location types (e.g., cafes, venues, parks) they have posted notes at.
- FR 9.7: The user shall be able to view the number of reactions their notes have received toward badge thresholds.
- FR 9.8: The user shall be able to display earned badges on their public profile.

User

- FR 10.4: The user shall be able to upload a profile picture.
- FR 10.5: The user shall be able to delete their account.

Mockups and Storyboards

Storyboard 0: Sign in/On boarding



A user with an associated user_id can sign into the Vibe Check system with their email and password.

Storyboard 1: Exploring nearby restaurants

Select Location Pin

Vibecheck - Restaurants

(map/heatmap)

Pin

Mario's Bistro

Buzzing

8 notes

0.2 mi away

Home Location Cloud

View Note Thread

Mario's Bistro

0.2 miles away

flect117 • 5 min ago

"Time around the block?"

12 3 replies

+ create Note

John taps a restaurant pin.
A preview card appears
showing a name, vibe
level, note count,
and distance.

John reads notes and replies confirming
the bng wait.
He goes back and selects a different
restaurant pin and sees positive notes.

Create New Note

← Create Note

Tony's Diner

Vibe Level:

☐ Dead ☐ quiet

☐ Buzzing ☒ moderate

Quick service!

category: [Restaurants ▼]

Post

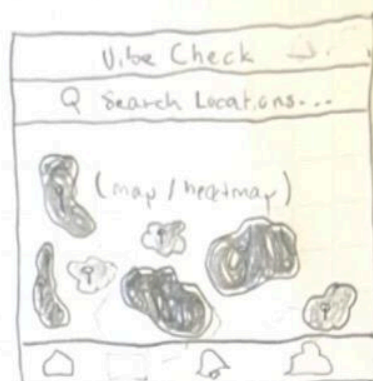
After eating, John selects
his vibe level, writes his
note, and taps Post
Note is published for future
users to see.

Storyboard 1.

Get current location



view Heat map



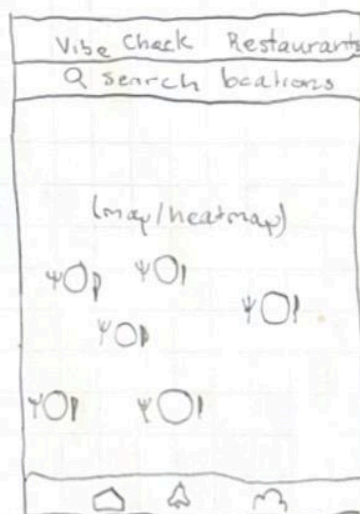
John allows location access.

Filter by Category



Heatmap loads showing activity levels nearby.

View Nearby Location Pins



John taps filter icon.
Selects "Restaurants".
Taps Apply.

Map now shows only restaurant pins
in the area.

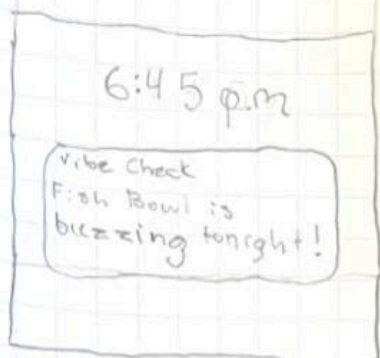
John browses nearby options

Users can use the heatmap to explore various locations. In addition, they can filter the heatmap to only show certain locations such as restaurants.

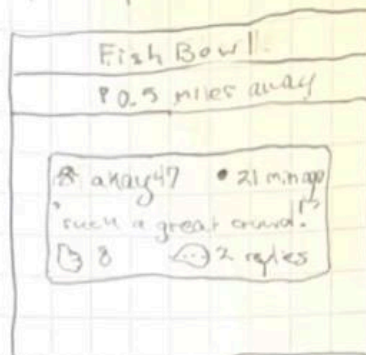
Storyboard 2: Push notifications

Storyboard 2.

Receive Push Notification



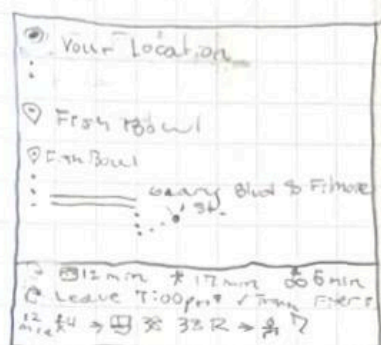
Open App to Location Detail



Chris receives a push notification that a venue near him is buzzing tonight.

The app opens to the note thread. Chris reads the notes.

Navigate to Location



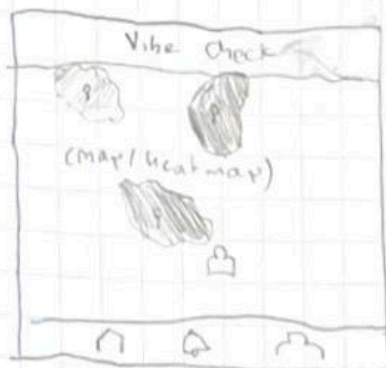
Chris taps directions to the venue

Users can get push notifications sent to their device letting them know if a location near them is buzzing.

Storyboard 3: Friends

Storyboard 3

Friend indicator on pin



Sue views the heatmap
and notices a friend
indicator on a pin.

Read friend's note



=>

Users can see their friends on the heatmap. Users can also see their friends' notes by clicking their icon on the map.

Storyboard 4: Badges

Storyboard 4

Post note



Bruno posts his 5th note at a Study Spot.



System detects threshold met
Badge unlock notification

View badges



Bruno can see his new badge on his profile as well as his other badges and recent notes.

Users can receive different badges the more they interact with the map.

System Design

Database Architecture

Backend Architecture

Network and Deployment Design

High Level APIs and Algorithms

Key Project Risks

Skills Risks

Risk: Team lacks expertise in required technologies (e.g., a new framework, cloud platform, or Database). **Mitigation:** Conduct a skills gap assessment early or schedule targeted training. Pair junior devs with senior leads on high-risk modules.

Schedule Risks

Risk: Underestimated task complexity, scope creep, or dependency delays (e.g., waiting on third-party APIs or another team's deliverable). **Mitigation:** Use time-buffered sprint planning (add 20–30% buffer to estimates). Define a formal change request process to control scope. Identify critical path dependencies upfront and monitor them weekly.

Technology Risks

Risk: Chosen technology proves unstable, deprecated, or incompatible with existing systems mid-project. **Mitigation:** Run a time-boxed technical spike/proof-of-concept before committing. Prefer well-supported, battle-tested libraries. Document fallback technology options before build begins.

Teamwork Risks

Risk: Poor communication, unclear ownership, or interpersonal conflict causing bottlenecks or duplicated work. **Mitigation:** Establish a RACI matrix (Responsible, Accountable, Consulted, Informed) at kickoff. Hold regular standups and retrospectives. Use a shared project tracker (Jira, Linear, etc.) to keep ownership visible.

Legal & Compliance Risks

Risk: Use of open-source libraries with incompatible licenses (e.g., GPL in a commercial product), data privacy violations (GDPR, CCPA), or IP ownership ambiguity with contractors. **Mitigation:** Audit all third-party dependencies for license compatibility early. Involve legal counsel for data handling design decisions. Ensure contractor agreements include clear IP assignment clauses.

Project Management

<https://arkha.atlassian.net/jira/software/projects/KAN/boards/1>.

Team Contributions

- Harry Kakadiya (Team Lead)
 - Version 1 Contributions:
- Kaitlyn Ashby (GitHub Master, Technical Writer)
 - Version 1 Contributions:
- Rahul Srinath (Software Architect)
 - Version 1 Contributions:
- Amulya Sagi (Backend Lead)
 - Version 1 Contributions:
- Aljhay Soriano (Scrum Master)
 - Version 1 Contributions:

Member	Score

Harry	
Kaitlyn	
Rahul	
Amulya	
Aljhay	